

# Implementierung und Evaluation eines robusten Spurerkennungsalgorithmus auf dem Duckiebot

\*Basierend auf dem Ansatz von Sang & Norris (2024)

Lukas Graf  
Informatik  
Hochschule Ruhr West  
Bottrop, Deutschland

Johannes Dulisch  
Informatik  
Hochschule Ruhr West  
Bottrop, Deutschland

Oliver Jockel  
Informatik  
Hochschule Ruhr West  
Bottrop, Deutschland

Nico Jurek  
Informatik  
Hochschule Ruhr West  
Bottrop, Deutschland

**Zusammenfassung**—Autonome mobile Roboter benötigen eine robuste Umgebungswahrnehmung, um auch unter widrigen Sichtbedingungen sicher zu navigieren. Bestehende Ansätze zur Spurerkennung, die primär auf statischen Kantenfiltern und der Hough-Transformation basieren, zeigen unter Laborbedingungen gute Ergebnisse, versagen jedoch bei visuellen Störungen wie Nebel oder diffusem Licht. Diese Arbeit stellt einen erweiterten Ansatz („Sang & Norris-Pipeline“) für die Duckiebot-Plattform vor, der eine adaptive Region of Interest (ROI), Fuzzy-Canny-Kantenerkennung und Kalman-Filter-Tracking kombiniert. Die Evaluation erfolgte durch vergleichende Experimente zwischen dem Referenzverfahren und dem neuen Ansatz unter simulierten Nebelbedingungen und Dunkelheit. Die Ergebnisse belegen, dass der neue Algorithmus die Detektionsrate bei starker Sichtbehinderung signifikant steigern konnte. Während das Referenzverfahren bei kritischen Störgrößen einen funktionalen Totalausfall erleidet, ermöglichte der hier vorgestellte Ansatz eine stabile und kontinuierliche Spurführung. Die Arbeit demonstriert somit, dass die Integration temporaler Filterung die Robustheit kostengünstiger Robotik-Systeme signifikant erhöht.

**Index Terms**—Autonomous Driving, Lane Detection, Fuzzy Logic, Duckiebot, Image Processing

## I. EINLEITUNG

Die Zuverlässigkeit von Perzeptionsalgorithmen ist der limitierende Faktor für die Sicherheit autonomer Fahrzeuge. Während die Detektion von Fahrbahnmarkierungen unter idealen Bedingungen als gelöst gilt, stellen reale Wetterphänomene wie Nebel oder starker Regen weiterhin eine Herausforderung für kamerabasierte Systeme dar.

Einfache algorithmische Ansätze, wie sie oft auf ressourcenbeschränkten Plattformen eingesetzt werden, basieren häufig auf statischen Schwellenwerten. Voruntersuchungen zeigten, dass diese Systeme bereits bei geringen Sichtbehinderungen versagen. Daraus leitet sich die Forschungsfrage dieser Arbeit ab: Wie kann eine Spurerkennungs-Pipeline auf begrenzter Hardware so optimiert werden, dass sie auch unter Störeinflüssen robust bleibt?

Ziel der Arbeit ist die Entwicklung und Validierung einer verbesserten Software-Pipeline („Sang & Norris“), die klassische Bildverarbeitung mit Methoden der Fuzzy-Logik und temporalen Filtern (Kalman) verknüpft. Damit soll die Lücke

zwischen einfachen Lehrbeispielen und robusten, sicherheitskritischen Anwendungen geschlossen werden.

## II. STAND DER TECHNIK

Die automatisierte Spurerkennung und -verfolgung ist seit Jahrzehnten ein zentraler Forschungsgegenstand im Bereich der Fahrerassistenzsysteme (ADAS) und der autonomen mobilen Robotik. Die wissenschaftliche Literatur unterscheidet hierbei primär zwischen klassischen, merkmalsbasierten Verfahren, probabilistischen Filtermethoden und modernen, datengetriebenen Deep-Learning-Ansätzen. Im Folgenden werden diese Paradigmen analysiert und im Hinblick auf ihre Anwendbarkeit auf ressourcenbeschränkten Plattformen bewertet.

### A. Merkmalsbasierte Verfahren und Vorverarbeitung

Der Industriestandard für ressourceneffiziente Systeme basiert traditionell auf der Extraktion geometrischer Primitive aus dem Kamerabild. Bradski und Kaehler [1] beschreiben die Hough-Transformation als robustes Mittel zur Detektion von parametrisierbaren Formen, insbesondere Geraden, in binären Kantenbildern. Die Vorverarbeitung erfolgt hierbei typischerweise durch Gradientenverfahren wie den Canny-Kantendetektor. Bao et al. [2] weisen in ihren Untersuchungen jedoch darauf hin, dass die Abhängigkeit von festen Gauß-Kernel-Größen und statischen Schwellenwerten (Thresholds) zu signifikanten Problemen führen kann. Insbesondere bei verrauschten Bildern, wechselnden Texturen oder diffusem Licht werden oft irrelevante Kanten detektiert oder schwache Fahrbahnmarkierungen übersehen.

### B. Robustheit durch Bildverbesserung und Style Transfer

Ein wesentlicher Forschungszweig beschäftigt sich mit der algorithmischen Aufbereitung des Rohbildes, um die Robustheit gegenüber Umweltfaktoren zu erhöhen. Son et al. [3] stellen hierfür einen Ansatz vor, der speziell für nächtliche Bedingungen entwickelt wurde. Sie nutzen bilaterale Filter zur Rauschunterdrückung unter Erhalt der Kanteninformationen und kombinieren dies mit einer „Simulated Annealing Gain Control“ (SAGC), um den lokalen Kontrast dynamisch zu optimieren. Solche Verfahren zeigen deutlich, dass reine

Kantendetektion ohne adaptive Vorverarbeitung bei schlechter Sicht unzureichend ist [3].

Noch einen Schritt weiter gehen Ko et al. [4] mit dem Einsatz generativer neuronaler Netze (GANs). Ihr Ansatz nutzt „CycleGANs“, um Bilder, die unter schlechten Lichtbedingungen aufgenommen wurden, mittels Style Transfer in synthetische Tagaufnahmen zu transformieren. Diese Methode des „Data Enhancement“ ermöglicht es nachgelagerten Algorithmen, auf quasi idealen Bildern zu arbeiten [4]. Obwohl diese Deep-Learning-basierten Ansätze beeindruckende Ergebnisse liefern, übersteigt ihre rechnerische Komplexität typischerweise die Kapazitäten eingebetteter Mikrocontroller, wie sie im Duckiebot verbaut sind. Unsere Arbeit verfolgt daher das Ziel, eine ähnliche Robustheit durch effizientere, regelbasierte Methoden (Fuzzy-Logik) zu erreichen, ohne auf GPU-Beschleunigung angewiesen zu sein.

### C. Integration in Navigationssysteme und Sensorfusion

Die Spurerkennung dient in modernen Systemen selten als alleinstehende Komponente, sondern wird oft mit anderen Lokalisierungsmethoden fusioniert. Lee et al. [5] demonstrieren ein integriertes Navigationssystem, das Spurerkennung zur Korrektur der lateralen Position nutzt, insbesondere in Umgebungen, in denen GPS-Signale unzuverlässig sind („GPS-denied environments“). Sie zeigen, dass die Kombination aus Odometrie (Dead Reckoning) und visueller Spurinformaton Driftfehler effektiv kompensieren kann [5].

Ähnliche Herausforderungen beschreiben Forschungen zur Lokalisierung bei Schnee und Regen [6]. Hier führen visuelle Störungen oft zum Verlust von Feature-Punkten, was die reine visuelle Odometrie instabil macht. Die Autoren schlagen daher vor, die Unsicherheit der visuellen Messung explizit zu modellieren und in den Fusionsfilter einzubinden [6]. Unser Ansatz greift diese Konzepte der Unsicherheitsmodellierung auf: Anstatt eines komplexen Multi-Sensor-Systems implementieren wir einen Kalman-Filter, der die visuelle Messung (Spurposition) mit einem Bewegungsmodell fusioniert. Dies erlaubt es dem System, ähnlich wie bei Lee et al., kurzzeitige Ausfälle der visuellen Wahrnehmung (z.B. durch dichten Nebel) mittels Prädiktion zu überbrücken.

### D. Probabilistische Stabilisierung vs. End-to-End Learning

Um Fehldetektionen („Geisterlinien“) zu minimieren, setzen fortschrittliche Systeme auf Tracking-Algorithmen. Kim et al. [7] schlagen eine probabilistische Glättung vor, um eine verlässliche Geometrie-Karte der Fahrspur zu erstellen, selbst wenn der Detektor unsichere Daten liefert. Wang et al. [8] nutzen B-Snake-Modelle, die Fahrbahnmarkierungen nicht als starre Linien, sondern als deformierbare Splines betrachten.

Im Gegensatz dazu stehen End-to-End Deep-Learning-Modelle wie YOLOv12 [9], die das gesamte Bild in einem Schritt analysieren. Während Terven et al. die überlegene semantische Segmentierungsleistung solcher Netze hervorheben, bleibt der „Black-Box“-Charakter und der hohe Ressourcenbedarf ein Nachteil für sicherheitskritische Echtzeitanwendungen auf Kleinsthardware. Zusammenfassend lässt sich feststellen,

dass eine Lücke im Stand der Technik existiert: Zwischen simplen Lehrbuch-Algorithmen (Canny/Hough) und High-End-KI-Systemen (GANs/YOLO) fehlt es oft an robusten, aber leichtgewichtigen Zwischenlösungen. Der in dieser Arbeit vorgestellte „Sang & Norris“-Ansatz adressiert genau diese Lücke durch die hybride Nutzung von Fuzzy-Logik und Kalman-Filterung.

## III. METHODIK

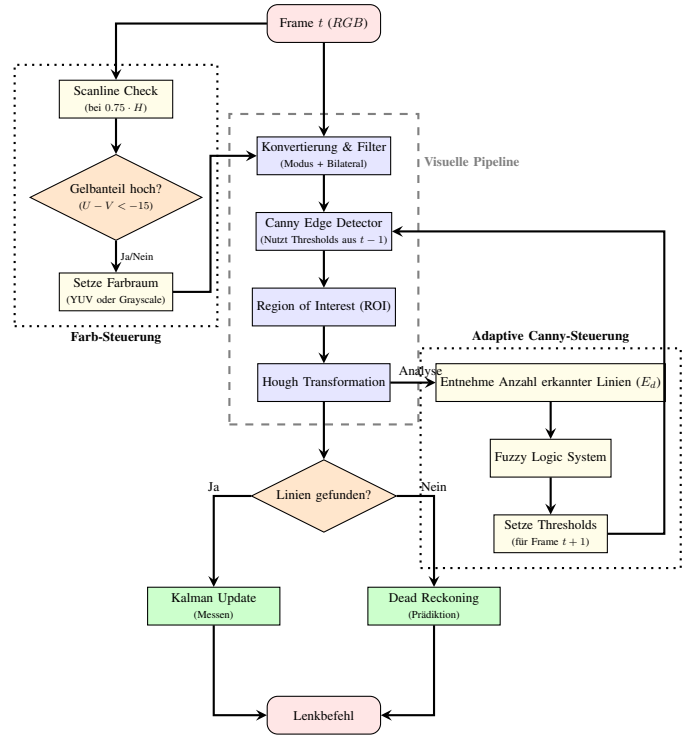


Abbildung 1. Vollständige Systemarchitektur. Links: Die **Farb-Steuerung** wählt adaptiv den Farbraum (YUV bei Gelb, sonst Grau). Mitte: Die eigentliche Bildverarbeitung. Rechts: Die **Adaptive Canny-Steuerung** optimiert die Kantendetektion. Unten: Das **Tracking-Modul** stabilisiert die Lenkbefehle.

### A. Referenzsystem (Baseline: CompareMethod)

Um die Leistung des adaptiven Ansatzes bewerten zu können, wurde zunächst ein klassischer, statischer Algorithmus implementiert (im Folgenden „CompareMethod“). Dieser orientiert sich an gängigen Implementierungen der Literatur für einfache Spurhalteassistenten und verwendet feste Parameter, die empirisch unter Laborbedingungen ermittelt wurden.

Die Verarbeitungskette der Baseline ist wie folgt definiert:

- **Vorverarbeitung:** Das Eingangsbild wird statisch in Graustufen konvertiert und mit einem Gaußschen Weichzeichner ( $7 \times 7$  Kernel,  $\sigma = 1.5$ ) geglättet.
- **Kantendetektion:** Ein Canny-Edge-Detektor arbeitet mit fixierten Schwellenwerten von  $T_{low} = 25$  und  $T_{high} = 75$ . Diese niedrigen Werte sind notwendig, um auch schwache Linien zu finden, führen jedoch bei Rauschen zu vielen Fehldetektionen.

- **Region of Interest (ROI):** Es wird ein statisches Rechteck verwendet, das den oberen Bildbereich (45 %) ausblendet. Im Gegensatz zum adaptiven Ansatz reagiert dieses ROI nicht auf Kurvenfahrten.

---

**Algorithm 1** Referenzsystem (Baseline)

---

**Require:**  $I_{raw}$  (Kamerabild RGB)

**Ensure:**  $L_{out}$  (Lenkwinkel / Linienposition)

```

1:  $I_{gray} \leftarrow \text{RGB2GRAY}(I_{raw})$ 
2:  $I_{blur} \leftarrow \text{GaussianBlur}(I_{gray}, \sigma = 1.5)$ 
3:  $I_{edge} \leftarrow \text{Canny}(I_{blur}, T_{low} = 25, T_{high} = 75)$ 
4:  $I_{roi} \leftarrow \text{ApplyRectangleMask}(I_{edge}, \text{Top} = 45\%)$ 
5:  $S_{lines} \leftarrow \text{HoughLinesP}(I_{roi})$ 
6:  $S_{filtered} \leftarrow \text{FilterByAngle}(S_{lines})$ 
7: if  $S_{filtered} \neq \emptyset$  then
8:    $L_{meas} \leftarrow \text{Average}(S_{filtered})$ 
9:    $L_{out} \leftarrow \text{KalmanCorrection}(L_{meas})$ 
10: else
11:    $L_{out} \leftarrow \text{KalmanPrediction}()$ 
12: end if
13: return  $L_{out}$ 

```

---

### B. Lösungsansatz (Sang & Norris-Pipeline)

Der in dieser Arbeit entwickelte Ansatz zielt darauf ab, die Schwächen klassischer, statischer Verfahren durch adaptive Parameteranpassung und temporale Filterung zu kompensieren. Die Pipeline kombiniert Methoden der digitalen Bildverarbeitung mit Fuzzy-Logik und Zustandsschätzung. Der grundlegende Ablauf ist in Algorithmus 2 dargestellt und gliedert sich in sechs logische Module (A–F), die im Folgenden detailliert beschrieben werden.

---

**Algorithm 2** Adaptive Pipeline (Sang & Norris)

---

**Require:**  $I_{raw}$  (RGB),  $State_{t-1}$  (Historie)

**Ensure:**  $L_{out}$  (Steuerung)

```

1: Modul A: Color Space Scaling
2: if  $\text{ScanlineCheck}(I_{raw}, y = 0.75H) == \text{YELLOW}$  then
3:    $I_{proc} \leftarrow \text{RGB2YUV}(I_{raw})$ 
4: else
5:    $I_{proc} \leftarrow \text{RGB2GRAY}(I_{raw})$ 
6: end if
7: Modul B: Noise Reduction
8:  $I_{smooth} \leftarrow \text{BilateralFilter}(I_{proc})$ 
9: Modul C: Fuzzy Edge Detection
10:  $T_{canny} \leftarrow \text{FuzzyInference}(\text{LineCount}_{t-1})$ 
11:  $I_{edge} \leftarrow \text{Canny}(I_{smooth}, T_{canny})$ 
12: Modul D: ROI Selection
13:  $VP \leftarrow \text{EstimateVanishingPoint}(State_{t-1})$ 
14:  $I_{roi} \leftarrow \text{ApplyDynamicTriangle}(I_{edge}, VP)$ 
15: Modul E & F: Detection & Tracking
16:  $S_{lines} \leftarrow \text{HoughLines}(I_{roi})$ 
17:  $\text{LineCount}_t \leftarrow \text{Count}(S_{lines})$ 
18:  $L_{out} \leftarrow \text{KalmanFilter}(S_{lines})$ 
19: return  $L_{out}$ 

```

---

1) *Farbraum-Transformation und adaptive Farbwahl:* Der Algorithmus arbeitet standardmäßig auf Graustufenbildern, um die Rechenlast gering zu halten. Um jedoch auch gelbe Fahrbahnmarkierungen bei schlechtem Kontrast sicher zu detektieren, implementiert das System einen adaptiven Wechsel in den YUV-Farbraum, basierend auf der Methode von Sang & Norris [10].

Hierzu wird eine horizontale **Scanline-Heuristik** auf 75 % der Bildhöhe ( $h_{scan} = 0.75 \cdot H$ ) angewendet. Entlang dieser Linie werden 11 Stützstellen im Abstand von je 1 % der Bildbreite gesampelt. Für jeden Punkt wird geprüft, ob er die spektrale Signatur einer gelben Markierung aufweist. Dies ist definiert durch die Differenz der Chrominanzkanäle  $U$  und  $V$ :

$$U(x) - V(x) < T_{yellow} \quad \text{mit } T_{yellow} = -15 \quad (1)$$

Wird diese Bedingung auf einer Seite des Fahrzeugs erfüllt (Indikator für gelbe Linie), wird für die weitere Verarbeitung dieser Bildhälfte der YUV-Farbraum statt des Graustufenbildes verwendet, da der Kontrast zwischen gelber Farbe und Asphalt im YUV-Raum signifikant höher ist. Die Umrechnung erfolgt gemäß:

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.169 & -0.331 & 0.500 \\ 0.500 & -0.419 & -0.081 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} \quad (2)$$

Um Fehlinterpretationen bei gelblichem Umgebungslicht (z. B. Natriumdampflampen in Tunneln) zu vermeiden, wird diese Funktion deaktiviert, falls mehr als 15 % des Gesamtbildes als „gelb“ klassifiziert werden.

a) *Rauschunterdrückung mittels Bilateralem Filter:* Vor der Kantendetektion wird das Bild geglättet. Im Gegensatz zum linearen Gauß-Filter, der das Bild  $I$  lediglich basierend auf dem geometrischen Abstand faltet und somit Kanten global unscharf macht, berücksichtigt der hier eingesetzte **Bilaterale Filter** zusätzlich die photometrische Ähnlichkeit der Pixel. Der Filterwert an der Position  $x$  berechnet sich aus zwei Gewichtungsfunktionen:

$$I_{new}(x) = \frac{1}{W_p} \sum_{x_i \in \Omega} I(x_i) f_r(\|I(x_i) - I(x)\|) g_s(\|x_i - x\|) \quad (3)$$

Hierbei gewichtet  $g_s$  den räumlichen Abstand (Spatial Domain) und  $f_r$  den Intensitätsunterschied (Range Domain). An starken Kanten (Fahrbahnmarkierung zu Asphalt) ist die Intensitätsdifferenz groß, weshalb der Term  $f_r$  gegen Null tendiert. Die Glättung wird an dieser Stelle effektiv gestoppt. Dies bewahrt die Kantensteilheit („Edge Sharpness“), während Rauschen in homogenen Flächen (Asphalt) reduziert wird. Dies ist entscheidend, um bei diffusem Licht (z. B. Nebel) die Kanteninformationen für die nachfolgende Detektion zu erhalten.

2) *Adaptive Kantendetektion mittels Fuzzy-Logik:* Ein zentrales Problem statischer Canny-Detektoren ist die Wahl fester Schwellenwerte ( $T_{low}, T_{high}$ ). Sinken die Bildgradienten wetterbedingt ab, werden relevante Kanten ignoriert. Unser

Ansatz implementiert ein Fuzzy-Inferenz-System (FIS), das diese Schwellenwerte dynamisch anpasst.

a) *FIS-Design und Regelbasis:* Als linguistische Eingangsvariable dient die **Anzahl der detektierten Hough-Linien** ( $L_h$ ) aus dem vorherigen Frame. Die Logik folgt dabei dem Prinzip: Wenn die Anzahl detektierter Linien signifikant sinkt (Indikator für Nebel/Sichtverlust), senkt das FIS die Schwellenwerte ab, um die Sensitivität des Detektors zu erhöhen.

Die Fuzzifizierung unterteilt  $L_h$  in fünf linguistische Mengen (Membership Functions). Die Regelbasis wurde basierend auf [10] definiert und für die niedrigeren Auflösungen des Duckiebots angepasst (siehe Tabelle I und Abb. 2).

Tabelle I  
FUZZY-REGELBASIS UND PARAMETER FÜR DIE CANNY-ADAPTION

| Mitgliedsfunktion | Wertebereich ( $L_h$ ) | Anpassung ( $\Delta T_{high}$ ) |
|-------------------|------------------------|---------------------------------|
| Too few           | $L_h \in [2, 45]$      | -1,5 (Minus some)               |
| Little few        | $L_h \in [40, 60]$     | -0,5 (Minus a little)           |
| Good              | $L_h \in [50, 60]$     | 0,0 (Zero)                      |
| Little many       | $L_h \in [50, 80]$     | +0,5 (Add a little)             |
| Too many          | $L_h \in [70, 1000]$   | +1,5 (Add some)                 |

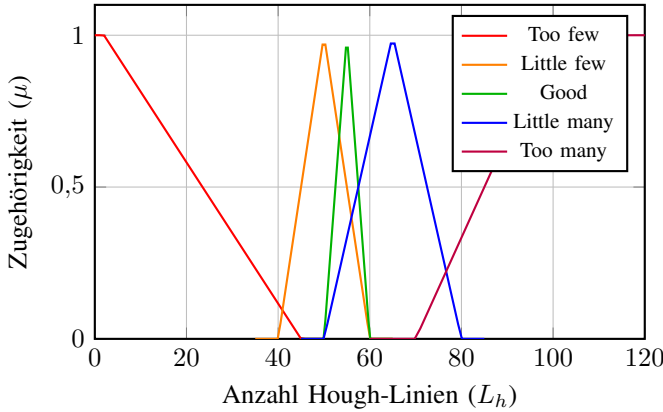


Abbildung 2. Zugehörigkeitsfunktionen der Eingangsvariable  $L_h$ .

*Anmerkung zur Parametrierung:* Während im Originalansatz von Sang & Norris [10] aufgrund hochauflösender Eingabebilder sehr hohe Linienanzahlen (20.000 – 30.000) als Referenz dienen, weichen die auf dem Duckiebot ermittelten Werte signifikant davon ab ( $L_h < 100$ ). Dies ist auf die geringere Sensorauflösung ( $640 \times 480$ ) zurückzuführen. Die Wertebereiche wurden daher empirisch neu kalibriert, um ein äquivalentes Filterverhalten auf der Embedded-Plattform zu gewährleisten.

b) *Defuzzifizierung und Adaption:* Die Defuzzifizierung berechnet aus den Mitgliedsgraden  $\mu_i$  und den Gewichten  $w_i$  einen diskreten Korrekturwert  $\Delta T$  mittels der gewichteten Durchschnittsmethode:

$$\Delta T = \frac{\sum_{i=1}^5 \mu_i(L_h) \cdot w_i}{\sum_{i=1}^5 \mu_i(L_h)} \quad (4)$$

Der Schwellenwert für den Canny-Filter wird anschließend iterativ aktualisiert ( $T_{high,new} = T_{high,old} + \Delta T$ ). Dies garantiert einen stetigen Übergang der Parameter und verhindert ein Flackern der Detektion.

3) *Adaptiver ROI (Region of Interest):* Um die Recheneffizienz zu steigern und Fehlerkennungen durch Hintergrundobjekte zu minimieren, verwendet der Algorithmus anstelle eines statischen Rechtecks eine **dreieckige Region of Interest**, deren Spitze ( $X_{ROI}, Y_{ROI}$ ) idealerweise im Fluchtpunkt der Fahrbahn liegt. Da sich die Lage des Fluchtpunkts durch Kurvenfahrten oder Nickbewegungen des Fahrzeugs ändert, werden die Koordinaten der ROI-Spitze dynamisch angepasst:

a) *Horizontale Anpassung ( $X_{ROI}$ ):* Die Standardposition liegt in der horizontalen Bildmitte. Wenn das Fahrzeug eine Kurve fährt und eine der beiden Fahrbahnmarkierungen aus dem Sichtfeld verliert, wird  $X_{ROI}$  um 5 % der Bildbreite in die Richtung der verlorenen Linie verschoben. Diese Verschiebung begrenzt das Risiko, dass die ROI bei engen Kurvenradien über den Fahrbahnrand hinausragt und dadurch relevante Bereiche des Bildes abschneidet.

b) *Vertikale Anpassung ( $Y_{ROI}$ ):* Die Standardhöhe der Spitze ist auf 2/3 der Bildhöhe festgelegt, um die Fahrbahn vollständig abzudecken. Im laufenden Betrieb wird dieser Wert basierend auf der Detektionsgüte optimiert:

- **Bei erfolgreicher Detektion:** Werden beide Fahrbahnlinien erkannt, wird die Höhe der ROI-Spitze auf das 1,1-fache der aktuellen Fluchtpunkthöhe gesetzt. Dies reduziert den Suchbereich auf das Nötigste.
- **Bei Detektionsausfall:** Wird eine oder beide Linien nicht erkannt, greift das System auf den Durchschnittswert der Fluchtpunkthöhen der letzten 30 erfolgreich verarbeiteten Frames zurück. Dies stabilisiert die ROI auch bei kurzzeitigem Sichtverlust.

4) *Temporale Stabilisierung (Kalman-Filter):* Hinsichtlich der temporalen Verfolgung unterscheidet sich der hier gewählte Ansatz von der Vorlage. Sang & Norris [10] merken an, dass Kalman-Filter bei fehlerhaften Detektionen zu ungenauen Prädiktionen führen können, weshalb sie bei einem Sichtverlust stattdessen direkt die Werte des vorangegangenen Frames ( $\rho, \theta$ ) beibehalten.

In dieser Arbeit kommt hingegen ein Kalman-Filter zum Einsatz. Dieser fusioniert die aktuelle Messung (Winkel  $\theta$ , Abstand  $d$ ) mit der Prädiktion aus dem vorherigen Zeitschritt. Dies dient dazu, das Signal zu glätten und bei kurzzeitigem Ausfall der visuellen Detektion (z. B. bei extrem dichtem Nebel) die Spurführung mittels Dead Reckoning aufrechtzuerhalten, bis wieder valide Messdaten vorliegen.

### C. Duckiebot Plattform

Als experimentelle Plattform dient der autonome Roboter "Duckiebotin der Konfiguration DB21. Das Chassis basiert auf einer Differential-Drive-Kinematik, angetrieben durch zwei DC-Motoren mit Hall-Effekt-Encodern zur Odometrie-Erfassung.

Die Umgebungswahrnehmung erfolgt primär über eine frontgerichtete Kamera mit einem 160°-Weitwinkelobjektiv

und einer Auflösung von 640×480, welche eine frühzeitige Erkennung von Fahrbahnmarkierungen auch in engen Kurvenradien ermöglicht. Ergänzend verfügt das System über eine IMU (Inertial Measurement Unit) sowie einen Time-of-Flight (ToF) Sensor zur Kollisionsvermeidung. [11]

Die Datenverarbeitung erfolgt auf einer eingebetteten Recheneinheit, dem NVIDIA Jetson Nano (4GB Version). Dieses System-on-Module (SoM) verfügt über eine 128-Kern Maxwell-GPU und einen Quad-Core ARM Cortex-A57 MP-Core Prozessor. [12]. Da der in dieser Arbeit vorgestellte Ansatz („Sang & Norris“) bewusst als leichtgewichtige algorithmische Pipeline konzipiert wurde, erfolgt die Berechnung ausschließlich auf der ARM-CPU. Dies demonstriert die Effizienz des Verfahrens, da die leistungsstarke Maxwell-GPU vollständig für potentielle parallele Aufgaben (z. B. neuronale Netze zur Objekterkennung) verfügbar bleibt [12]. Das System wird über eine 64 GB Micro-SD-Karte betrieben und durch einen 10 Ah Lithium-Polymer-Akku mit Spannungsüberwachung und Soft-Shutdown-Funktionalität versorgt [11].

#### D. Implementierung

Die Realisierung des Systems erfolgte in C++ unter Verwendung des *Robot Operating System* (ROS) [13] als Middleware sowie der Bildverarbeitungsbibliothek OpenCV. Die Architektur wurde unter den Gesichtspunkten der Modularität und Vergleichbarkeit entworfen. Im Folgenden werden die Systemumgebung, die Softwarearchitektur sowie die regelungstechnische Umsetzung detailliert.

1) *Systemumgebung und Containerisierung*: Um eine reproduzierbare und hardwareunabhängige Laufzeitumgebung zu gewährleisten, ist die gesamte Software in einem Docker-Container gekapselt. Als Basis dient das Duckietown-Image `dt-ros-commons` [14], welches die notwendigen ROS-Bibliotheken bereitstellt. Der entwickelte Code agiert als eigenständiger ROS-Node (`driver_cpp_node`). Die Kommunikation mit der Hardware erfolgt über das *Publish-Subscribe*-Pattern von ROS:

- **Input:** Der Node abonniert das Kamerabild über das Topic `/camera_node/image/compressed`.
- **Output:** Steuerbefehle werden auf das Topic `/wheels_driver_node/wheels_cmd` publiziert. Hierbei wird nicht ein einzelner Lenkwinkel übertragen, sondern es werden spezifische Geschwindigkeitswerte für das linke und rechte Rad (`vel_left`, `vel_right`) gesendet, die aus der Regelabweichung und einer globalen Sollgeschwindigkeit berechnet werden.

2) *Softwarearchitektur und Modularität*: Ein zentraler Aspekt der Implementierung ist die strikte Trennung zwischen Bildverarbeitung und Fahrzeugregelung. Um die Vergleichbarkeit der beiden Verfahren („CompareMethod“ und „Sang & Norris-Pipeline“) zu gewährleisten, wurde eine einheitliche Schnittstellendefinition implementiert. Beide Verfahren akzeptieren ein rohes Eingangsbild (`cv::Mat`) und liefern als Rückgabewert einen Vektor aus abstrahierten Spurobjekten (`std::vector<LaneLine>`).

Dies ermöglichte die Umsetzung einer *Conditional Compilation* mittels Präprozessor-Direktiven (`#ifdef USE_NEW_PIPELINE`) in der `main.cpp`. Dadurch kann das Verfahren zur Kompilierzeit ressourcenschonend ausgetauscht werden, ohne die Regelungslogik anpassen zu müssen.

a) *Pipeline-Architektur (Sang & Norris Methode)*: Die grundlegende Architektur der Implementierung adaptiert die im Referenzpaper [10] vorgestellte Struktur, um eine wissenschaftliche Fundierung zu gewährleisten. Jedes logische Teilstück des Algorithmus wurde in ein eigenständiges C++ Modul überführt. Bei der konkreten Ausgestaltung weicht die Implementierung jedoch bewusst von der Vorlage ab, um das Verfahren an die spezifischen Hardware-Gegebenheiten des Duckiebots anzupassen:

- 1) Empirische Parametrierung: Die Schwellenwerte der Fuzzy-Edge-Detection sowie die geometrischen Parameter der ROI-Auswahl wurden nicht identisch aus dem Paper übernommen, sondern empirisch an die Kameraoptik und die Perspektive des Roboters angepasst.
- 2) Erweitertes Tracking: Während das Referenzpaper den Kalman-Filter lediglich als optionale Erweiterung vorgeschlägt, wurde dieser in unserer Implementierung aufgrund verbesserter Testergebnisse fest integriert.

Die Datenverarbeitung erfolgt dementsprechend sequenziell über sechs Stufen:

- Modul A: Color Space Scaling (Farbraumanpassung)
- Modul B: Noise Reduction (Rauschunterdrückung)
- Modul C: Fuzzy Canny Edge Detection (Adaptive Kantenerkennung mit angepasster Regelbasis)
- Modul D: ROI Selection (Hardware-spezifische Interessensbereichsauswahl)
- Modul E: Line Detection (Hough-Transformation)
- Modul F: Tracking (Kalman-Filterung zur Stabilisierung)

3) *Regelungstechnische Umsetzung*: Die Querregelung des Fahrzeugs erfolgt durch einen PID-Regler, der die Abweichung zwischen der berechneten Spurmitte und der Bildmitte minimiert. Die Implementierung nutzt eine Signalglättung (*Exponential Moving Average Filter*), um die Fahrstabilität zu erhöhen.

Da das Referenzverfahren („CompareMethod“) im Gegensatz zur Pipeline über keinen integrierten Kalman-Filter verfügt, wirken sich Bildrauschen oder Fehldetektionen unmittelbar auf die Stellgröße aus. Dies kann zu abrupten Lenkbewegungen und einem Verlassen der Spur führen. Um dies zu verhindern, wird der berechnete Fehlerwert ( $error_{raw}$ ) vor der PID-Verarbeitung geglättet:

$$error_{smooth} = \alpha \cdot error_{raw} + (1 - \alpha) \cdot error_{prev} \quad (5)$$

Mit einem empirisch ermittelten Faktor für  $\alpha$  werden Ausreißer im Referenzverfahren effektiv gedämpft. Für die Sang & Norris-Pipeline wäre dieser Schritt aufgrund des internen Trackings technisch nicht zwingend erforderlich, schadet

der Regelgüte jedoch nicht und erlaubt die Nutzung eines einheitlichen Reglers für beide Verfahren.

Der vom PID-Regler berechnete Stellwert (Lenkintensität) wird anschließend in eine **Differential-Drive-Steuerung** umgesetzt. Dabei wird der Lenkwert mit einer konstanten Vorwärtsgeschwindigkeit verrechnet. Ein Lenkwert von 0 resultiert in identischen Geschwindigkeiten für beide Räder (Geradeausfahrt), während positive oder negative Werte eine Geschwindigkeitsdifferenz erzeugen und das Fahrzeug somit in eine Kurve zwingen.

#### IV. EXPERIMENT

##### A. Versuchsaufbau und Methodik

Um die Leistungsfähigkeit der beiden Verfahren („Compare-Method“ vs. „Sang & Norris-Pipeline“) objektiv vergleichen zu können, wurde das Experiment in zwei getrennte Testkonzepte unterteilt: eine quantitative Analyse der reinen Algorithmik und eine qualitative Bewertung des dynamischen Fahrverhaltens.

1) *Simulierte Umweltbedingungen (Gültig für alle Versuche)*: Um reproduzierbare Störgrößen zu generieren, wurden verschiedene Szenarien definiert. Sichtbehinderungen durch Nebel wurden mittels milchig-transparenter Folienschichten vor der Kameralinse simuliert, während Nachtfahrten durch gezielte Variation der Beleuchtung nachgestellt wurden:

- **Optimal**: Keine Sichtbehinderung bei normaler Raumbeleuchtung (Referenzwert).
- **Leichter Nebel**: 2 Schichten Folie (Reduzierter Kontrast, leichtes Rauschen).
- **Starker Nebel**: 4 Schichten Folie (Starker Kontrastverlust, diffuses Rauschen).
- **Dunkelheit (starke Beleuchtung)**: Starke, punktuelle Lichtquelle in abgedunkelter Umgebung (simuliert eigenes Licht durch Scheinwerfer oder Straßenlaternen). Testet die Robustheit gegen lokale Überbelichtung und extreme Kontraste.
- **Dunkelheit (schwache Beleuchtung)**: Minimale Restbeleuchtung. Die Fahrbahnmarkierungen sind visuell noch erkennbar, jedoch weist das Kamerabild aufgrund der hohen Sensor-Empfindlichkeit (High-ISO) ein extremes Bildrauschen auf. Dies testet die Grenzen der Kantenerkennung bei schlechtem Signal-Rausch-Verhältnis.

2) *Versuch A: Quantitative Code-Evaluation*: Dieser Test bewertet die Erkennungsleistung möglichst isoliert von mechanischen Einflüssen.

a) *Versuchsaufbau*: Der Roboter wurde auf einer 1,0 m langen, idealen Geraden mit einer Spurbreite von 22 cm platziert. Die Messdauer betrug 30 Sekunden pro Szenario bei fixiertem Winkel, um einen konstanten Datenstrom zu gewährleisten.

b) *Metriken*: Gemessen wurden die *Valid Detection Rate* (VDR) (Prozentsatz der Frames mit nahezu korrekter Erkennung der Spurbreite) sowie die *False Positive Rate* (Anteil der Falscherkennungen auf leerer Fläche).

3) *Versuch B: Dynamischer Fahrttest*: Hierbei wurde die Robustheit der Spurführung im komplexen Umfeld bewertet.

a) *Versuchsaufbau*: Ein Kurs mit 22 cm Spurbreite ( $\pm 4$  cm) wurde in vier Segmente unterteilt: Start (0,5 m), Störgröße (30 cm Unterbrechung durch eine Kreuzung), 180°-Kurve (80 cm Radius) und Zielauslauf.

b) *Durchführung*: Um stochastische Effekte (z. B. Sensorrauschen oder minimale Abweichungen im Startwinkel) zu mitteln, wurde jedes Szenario in mindestens drei separaten Durchläufen getestet. Die finale Bewertung repräsentiert das durchschnittliche Fahrverhalten des jeweiligen Verfahrens unter identischen Umweltbedingungen.

c) *Metrik*: Das Fahrverhalten wurde qualitativ auf einer Skala von 1–10 bewertet, unterteilt in die Kategorien *Kritisch* (1–3, Eingriff nötig), *Akzeptabel* (4–6, Pendeln) und *Sehr gut* (7–10, stabil).

##### B. Ergebnisse

1) *Quantitative Code-Evaluation (Versuch A)*: Die Ergebnisse der isolierten algorithmischen Auswertung sind in Tabelle II zusammengefasst.

Unter *idealen Sichtbedingungen* zeigte der statische Referenzansatz mit 71,5 % eine leicht höhere Erkennungsrate als die adaptive Pipeline (60,0 %). Dies ist darauf zurückzuführen, dass die festen Schwellenwerte der „CompareMethod“ exakt auf die optimalen Lichtverhältnisse kalibriert waren und somit weniger anfällig für Rauschen im Bildhintergrund sind.

Sobald jedoch visuelle Störungen auftraten, kehrte sich das Verhältnis drastisch um. Während die Valid Detection Rate (VDR) des statischen Verfahrens bereits bei leichtem Nebel massiv einbrach und bei starkem Nebel sowie bei *Dunkelheit mit starker Beleuchtung* einen Totalausfall verzeichnete, blieb der adaptive Ansatz stabil. Selbst unter diesen extremen Bedingungen (4 Schichten Folie bzw. Überbelichtung) konnte eine VDR von über 60 % aufrechterhalten werden. Dies belegt die Effektivität der Fuzzy-Logik, die den Canny-Threshold dynamisch an Kontrastverlust und Überbelichtung anpasst.

Eine physikalische Grenze erreichten beide Systeme hingegen im Testfall *Dunkelheit mit schwacher Beleuchtung*. Obwohl die Fahrbahnmarkierungen für das menschliche Auge noch schemenhaft erkennbar waren, fielen die Erkennungsraten beider Verfahren auf unter 1 %. Die Ursache liegt hier nicht in der absoluten Unsichtbarkeit der Linien, sondern im extremen *Bildrauschen*. Durch die automatische Belichtungsregelung der Kamera wurde der ISO-Wert (Gain) maximiert, was zu einem stark verrauschten Bildsignal führte (niedriges Signal-Rausch-Verhältnis). Die Kantenerkennungs-Algorithmen interpretierten dieses Rauschen als tausende mikroskopische Kanten, wodurch die eigentlichen Fahrbahnkonturen nicht mehr zuverlässig vom Hintergrundrauschen differenziert werden konnten.

Dieser Gewinn an Robustheit in den anderen Szenarien geht jedoch mit einer erhöhten *False Positive Rate* einher. Auf leeren Flächen detektierte der adaptive Ansatz in 13,9 % der Fälle fälschlicherweise Linien („Geisterlinien“), während das statische Verfahren mit 0,7 % fast keine Fehler machte. Das System erkaufte sich die Sichtbarkeit unter schwierigen



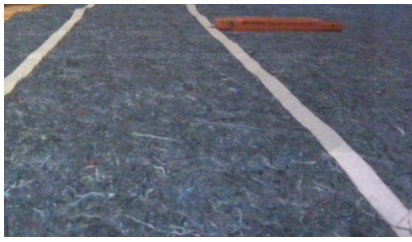


Abbildung 3. Optimal (Referenz)

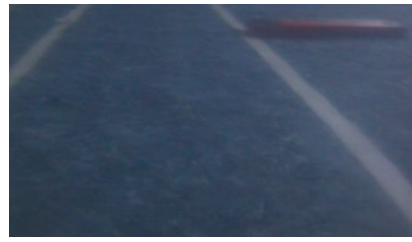


Abbildung 4. Leichter Nebel (2 Schichten)

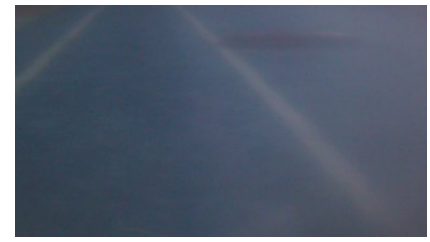


Abbildung 5. Starker Nebel (4 Schichten)



Abbildung 6. Dunkelheit (starke Beleuchtung)

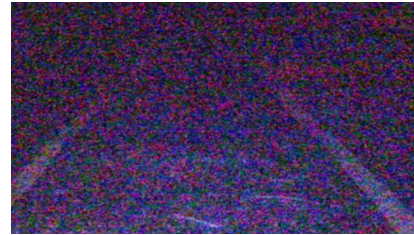


Abbildung 7. Dunkelheit (schwache Beleuchtung)

Abbildung 8. Vergleich der Sichtbedingungen aus der Roboterperspektive.

Bedingungen also durch eine höhere Sensitivität, die anfälliger für Bildrauschen ist.

Tabelle II  
QUANTITATIVE ERGEBNISSE (CODE-EVALUATION)

| Szenario                          | Statisch | Adaptiv | Interpretation                                                                                                                                   |
|-----------------------------------|----------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Optimal (Klar)                    | 71.5%    | 60.0%   | Statischer Ansatz erkennt bei Idealbedingungen häufiger valide Spuren.                                                                           |
| Leichter Nebel                    | 39.8%    | 73.0%   | Adaptiver Ansatz ist deutlich robuster gegen Rauschen.                                                                                           |
| Starker Nebel                     | 0.3%     | 72.0%   | Statisch versagt fast vollständig; Adaptiv bleibt stabil.                                                                                        |
| Dunkelheit (starke Beleuchtung)   | 2.5%     | 62.7%   | Adaptiv zeigt hohe Resistenz gegen lokale Überbelichtung.                                                                                        |
| Dunkelheit (schwache Beleuchtung) | 0.6%     | 0.7%    | Signal-Rausch-Problem: Spuren sind zwar vorhanden, gehen aber im massiven Sensorrauschen unter. Die Kantenerkennung versagt in beiden Verfahren. |
| Leerfläche (False Pos.)           | 0.7%     | 13.9%   | Adaptiv neigt eher zu „Halluzinationen“ (Geisterlinien).                                                                                         |

2) *Dynamischer Fahrtest (Versuch B)*: Die Ergebnisse der dynamischen Fahrtests sind in Tabelle III dargestellt. Es zeigt sich eine starke Diskrepanz zwischen den Verfahren bei zunehmender Sichtbehinderung.

Unter *optimalen Bedingungen* erreichten beide Verfahren akzeptable Werte, wobei der statische Ansatz bereits hier Schwächen bei Linienunterbrechungen (Kreuzungen) zeigte (Score 6), während der adaptive Ansatz stabil blieb (Score 7).

Bei *simuliertem Nebel* (Leicht bis Stark) brach die Leistung des Referenzverfahrens drastisch ein. Bereits bei leichtem Nebel war keine zuverlässige Spurführung mehr möglich (Score 2). Unter starken Nebelbedingungen versagte der statische Algorithmus vollständig (Score 0), da der Canny-Filter keine zusammenhängenden Kanten mehr extrahieren konnte. Im Gegensatz dazu ermöglichte die „Sang & Norris“-Pipeline auch bei starker Sichtbehinderung eine autonome Fahrt (Score 3–5). Zwar wurde ein erhöhtes Pendelverhalten (Oszillation) beobachtet, ein vollständiger Spurverlust ist jedoch in den Versuchen nur bis zu zwei Mal pro Durchlauf aufgetreten.

Auch bei *Dunkelheit mit starker Beleuchtung* bestätigte sich die Überlegenheit des adaptiven Ansatzes. Der statische Algorithmus neigte hier auf gerader Strecke zu starker Oszillation und verließ in Kurven systematisch die Spur (Score 3). Die „Sang & Norris“-Pipeline hingegen meisterte die Gerade präzise (Score 5) und konnte auch in Kurven die Spur halten, wenngleich hier erneut eine starke Tendenz zum Fahren am äußeren Rand beobachtet wurde.

## V. DISKUSSION UND FAZIT

### A. Diskussion

Die durchgeführten Experimente bestätigen die ursprüngliche Hypothese, dass eine adaptive Parameterwahl die Robustheit der Spurerkennung unter schlechten Sichtbedingungen signifikant steigern kann. Dennoch offenbaren die Ergebnisse einen klaren Zielkonflikt (Trade-off) zwischen Sensitivität und Präzision, der im Folgenden analysiert wird.

Der signifikanteste Befund dieser Arbeit ist die Diskrepanz der Valid Detection Rate (VDR) im Nebel-Szenario. Während das statische Referenzverfahren („CompareMethod“) bei starkem Nebel mit einer VDR von 0,3 % funktional vollständig ausfiel, konnte der adaptive Ansatz („Sang & Norris“)

Tabelle III  
QUALITATIVE BEWERTUNG DES DYNAMISCHEN FAHRVERHALTENS (SCORE 1–10)

| Wetter                             | Szenario | Verfahren | Score | Beobachtung & Verhalten                                                                                    |
|------------------------------------|----------|-----------|-------|------------------------------------------------------------------------------------------------------------|
| Optimal                            | Gerade   | Statisch  | 6     | Präzise auf der Geraden. Kritisch bei Kreuzungen: Falscherkennung führt oft zu sofortigem Spurverlust.     |
|                                    | Gerade   | Adaptiv   | 7     | Leichtes Pendeln, aber robustes Halten der Spur.                                                           |
|                                    | Kurve    | Statisch  | 6     | Hält Kurve meistens, fährt aber am äußeren Rand. Kritisch: Bei Fehlerkennung unkontrolliertes Drehen.      |
|                                    | Kurve    | Adaptiv   | 5     | Kein Pendeln, aber starke Tendenz zum äußeren Rand. Bei Spurverlust ebenfalls unkontrolliertes Drehen.     |
| Leichter Nebel                     | Gerade   | Statisch  | 2     | Instabil. Kreuzungen werden nicht bewältigt. Spur wird im Schnitt nur ca. 5 Sekunden gehalten.             |
|                                    | Gerade   | Adaptiv   | 6     | Verhalten ähnlich wie bei optimalen Bedingungen. Erhöhtes Pendeln an Kreuzungen, aber kein Spurverlust.    |
|                                    | Kurve    | Statisch  | 2     | Kurvenerkennung vorhanden, aber falsche Einschätzung des Lenkwinkels. Verlässt Spur nach wenigen Sekunden. |
|                                    | Kurve    | Adaptiv   | 5     | Keine signifikante Verschlechterung gegenüber optimalen Bedingungen.                                       |
| Starker Nebel                      | Gerade   | Statisch  | 0     | Totalausfall. Keine Spurführung möglich.                                                                   |
|                                    | Gerade   | Adaptiv   | 5     | Gute Spurführung. Gelegentliches Abkommen bei langen Kreuzungsbereichen ohne klare Markierung.             |
|                                    | Kurve    | Statisch  | 0     | Totalausfall.                                                                                              |
|                                    | Kurve    | Adaptiv   | 3     | Lange Kurven werden meist gehalten (starkes Pendeln). Bei engen Radien teilweise unkontrolliertes Drehen.  |
| Dunkelheit<br>(starke Beleuchtung) | Gerade   | Statisch  | 4     | Starkes Pendeln (Oszillation).                                                                             |
|                                    | Gerade   | Adaptiv   | 5     | Präzise. Kreuzung ohne Markierung unsicher.                                                                |
|                                    | Kurve    | Statisch  | 3     | Systematisches Abkommen (1x pro Kurve).                                                                    |
|                                    | Kurve    | Adaptiv   | 4     | Hält Spur (berührt Außenlinie).                                                                            |

eine VDR von 72,0 % aufrechterhalten. Dies belegt, dass die statische Festlegung von Canny-Schwellenwerten für reale Anwendungen unzureichend ist, da der globale Kontrastverlust bei Nebel die Bildgradienten unter diese starre Grenze drückt. Das implementierte Fuzzy-Logic-System konnte diesen Kontrastverlust erfolgreich kompensieren, indem es die Schwellenwerte dynamisch absenkte. Dies validiert den theoretischen Ansatz von Sang & Norris auch für die limitierte Hardware des Duckiebots.

Interessanterweise zeigte der statische Ansatz unter optimalen Bedingungen eine höhere Erkennungsrate (71,5 %) als der adaptive Ansatz (60,0 %). Dies lässt sich darauf zurückführen, dass die Parameter der „CompareMethod“ gezielt auf diese idealen Bedingungen angepasst waren. Der adaptive Ansatz hingegen erkaufte sich seine Robustheit durch eine erhöhte Sensitivität, was zu einer Anfälligkeit für Bildrauschen führt.

Dies manifestiert sich deutlich in der False Positive Rate auf leeren Flächen: Der adaptive Algorithmus detektierte in 13,9 % der Fälle „Geisterlinien“, wo keine waren, während das statische System (0,7 %) hier fast fehlerfrei blieb. Das System tendiert folglich zur Überinterpretation von Rauschmustern. Für eine Sicherheitsanwendung ist dies kritisch zu bewerten. Denn während ein False Negative, also ein Nichterkennen der Linien, zum Verlassen der Spur führt, kann ein False Positive, also die fälschliche Detektion nicht existenter Linien, zu plötzlichen, gefährlichen Lenkmanövern führen.

Eine harte physikalische Grenze wurde im Szenario „Dunkelheit mit schwacher Beleuchtung“ identifiziert. Hier versagten beide Ansätze mit Erkennungsraten unter 1 %. Die Ursachenanalyse muss hier über die reine Algorithmik hinaus

auf die Sensorik erweitert werden. Das automatische Gain Control der Duckiebot-Kamera führte zu einem massiven ISO-Rauschen, das vom Canny-Filter fälschlicherweise als Kanteninformation interpretiert wurde. Dies zeigt, dass selbst adaptive Kantendetektoren machtlos sind, wenn das Signal-Rausch-Verhältnis (SNR) einen kritischen Wert unterschreitet. Hier wäre eine Fusion mit aktiven Sensoren, die unabhängig vom Umgebungslicht arbeiten, notwendig.

Die qualitativen Fahrtests zeigten, dass eine stabile Spurerkennung (hohe Valid Detection Rate) nicht zwangsläufig in einem ruhigen Fahrverhalten resultiert. Obwohl die adaptive Pipeline im Nebel die Spur teilweise zuverlässig hielt (Score 3-6), wurde ein erhöhtes Pendeln (Oszillation) des Roboters beobachtet. Diese Instabilität ist primär auf die Regelungstechnik zurückzuführen. Für die Vergleichbarkeit der Experimente wurden die Parameter des PID-Reglers konstant gehalten und entsprachen der Abstimmung für das statische Referenzsystem. Da die adaptive Pipeline durch die zusätzlichen Filterstufen (Bilateral, Kalman) eine veränderte Signalcharakteristik und Dynamik aufweist, führte die Nutzung der unveränderten Regler-Parameter zu einem leichten Übersteuern. Das System reagierte auf kleine Abweichungen zu aggressiv, was sich in der beobachteten Oszillation äußerte. Dies verdeutlicht, dass die Einführung einer robusteren Bildverarbeitung („Sang & Norris“-Ansatz) auch eine Neu-Parametrierung des nachgelagerten Regelkreises erfordert, um die gewonnene Detektionssicherheit in ein glattes Fahrverhalten zu übersetzen.

Der „Sang & Norris“-Ansatz löst also das Problem der Sichtbarkeit (Detektion) bei schlechtem Wetter erfolgreich, jedoch entstehen dabei neue Herausforderungen im Bereich



der Signalqualität (False Positives) und der Systemdynamik.

### B. Fazit

Ziel der Arbeit war es zu prüfen, ob sich die Robustheit der Spurerkennung auf ressourcenbeschränkter Hardware durch die Adaptierung der Sang & Norris-Pipeline signifikant steigern lässt. Die Ergebnisse bestätigen, dass die Kombination aus Fuzzy-Logik-basierter Kantendetektion und temporalem Tracking (Kalman-Filter) den klassischen, statischen Ansätzen unter schlechten Sichtbedingungen deutlich überlegen ist.

Insbesondere in simulierten Nebelszenarien zeigte sich die Stärke des adaptiven Ansatzes: Während das Referenzverfahren („CompareMethod“) bei starkem Nebel mit einer Valid Detection Rate (VDR) von 0,3 % vollständig versagte, konnte der hier implementierte Algorithmus eine VDR von 72 % aufrechterhalten und eine autonome Fahrt ermöglichen. Dies belegt, dass die dynamische Anpassung der Canny-Schwellenwerte durch das Fuzzy-System effektiv auf den globalen Kontrastverlust reagiert.

Dennoch offenbarte die Evaluation klare physikalische und algorithmische Grenzen. Zum einen erkaufte sich das System die erhöhte Sensitivität mit einer gesteigerten Rate an „False Positives“ (Geisterlinien) auf leeren Flächen (13,9 % gegenüber 0,7 % beim Referenzverfahren). Zum anderen zeigte der Versuch bei „Dunkelheit mit schwacher Beleuchtung“, dass reine Bildverarbeitungsalgorithmen bei extremem Sensorrauschen an ihre Grenzen stoßen. Da das Signal-Rausch-Verhältnis hier zu ungünstig war, versagten beide untersuchten Verfahren gleichermaßen, was die Abhängigkeit von der Qualität der optischen Sensorik unterstreicht.

Für zukünftige Entwicklungen auf der Duckiebot-Plattform empfiehlt sich daher eine hybride Lösung: Ein Zustandsautomat könnte je nach ermittelter Sichtqualität zwischen dem präzisen statischen Verfahren (bei gutem Wetter) und dem robusten adaptiven Verfahren (bei Nebel) umschalten.

### LITERATUR

- [1] G. Bradski and A. Kaehler, *Learning OpenCV: Computer vision with the OpenCV library*. O'Reilly Media, Inc., 2008.
- [2] P. Bao, L. Zhang, and X. Wu, “Canny edge detection enhancement by scale multiplication,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 9, 2005.
- [3] J. Son, H. Yoo, S. Kim, and K. Sohn, “Robust nighttime road lane line detection using bilateral filter and sagc,” *IEEE Access*, 2019.
- [4] Y. Ko, S. Yun, and J. Jung, “Lane detection in low-light conditions using an efficient data enhancement: Light conditions style transfer,” *IEEE Transactions on Intelligent Transportation Systems*, 2021.
- [5] S. Lee, D. Kim, and K. Yoon, “An integrated vehicle navigation system utilizing lane-detection and lateral position estimation systems in difficult environments for gps,” *IEEE Transactions on Intelligent Transportation Systems*, 2015.
- [6] M. Aldibaja, N. Suganuma, and K. Yoneda, “Improving localization accuracy for autonomous driving in snow-rain environments,” *Advanced Robotics*, 2017.
- [7] J. Kim and et al., “Probabilistic smoothing based generation of a reliable lane geometry map with uncertainty of a lane detector,” *IEEE Access*, no. 7, 2022.
- [8] Y. Wang, E. K. Teoh, and D. Shen, “Lane detection and tracking using b-snake,” *Image and Vision Computing*, vol. 22, no. 4, pp. 269–280, 2004.
- [9] J. Terven and D. Cordova-Esparza, “A comprehensive review of yolo: From yolov1 to yolov8 and beyond,” *arXiv preprint arXiv:2304.00501*, 2023.

- [10] I.-C. Sang and W. R. Norris, “A robust lane detection algorithm adaptable to challenging weather conditions,” *IEEE Access*, vol. 12, pp. 11 185–11 195, 2024.
- [11] Duckietown, “Duckiebot (db-21) – diy autonomous car kit specifications,” <https://get.duckietown.com/products/duckiebot-db21>, n.d.
- [12] NVIDIA Corporation, *NVIDIA Jetson Nano System-on-Module Data Sheet*, 2025, version 1.1. Verfügbar unter: <https://developer.nvidia.com/embedded/jetson-nano>.
- [13] M. Quigley et al., “Ros: an open-source robot operating system,” in *ICRA workshop on open source software*, vol. 3, no. 3.2, 2009, p. 5.
- [14] L. Paull, J. Tani et al., “Duckietown: an open, inexpensive and flexible platform for autonomy education and research,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 1497–1504.